Introducción al Federated Learning (FL)

De aprendizaje centralizado a entrenamiento colaborativo con privacidad

Mineria de datos

Universidad de Murcia

17 de marzo de 2026

Objetivos de la sesión

- Entender el flujo clásico de ML **centralizado** y cómo se agrupan los datos.
- Identificar **riesgos de privacidad** y cumplimiento en el enfoque centralizado.
- Introducir **Federated Learning**: origen, esquema, entidades y componentes.
- Describir el **paso a paso** (entrenar $\rightarrow$ mandar $\rightarrow$ agregar $\rightarrow$ mandar).
- Clasificar FL por **tipologías**: cross-device/cross-silo, centralized/distributed, horizontal/vertical.
- Entender **desafíos clave** (heterogeneidad, privacidad, seguridad) y **mitigaciones**.

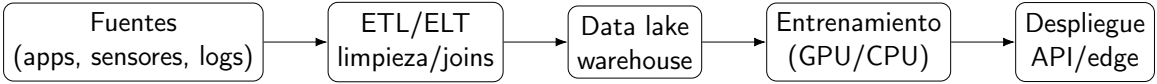
Agenda

1. Contexto: aprendizaje centralizado
2. Federated Learning: idea, origen y esquema
3. FL paso a paso: entrenar–mandar–agregar–mandar
4. Tipos de FL (taxonomías principales)
5. Desafíos en FL y posibles soluciones
6. Cierre: cuándo usar FL y qué viene después

¿Qué entendemos por ML centralizado?

- Datos de múltiples fuentes $\rightarrow$ se **centralizan** en un repositorio (data lake/warehouse).
- Un **servidor/cluster** entrena el modelo con acceso directo a los datos.
- Se despliega un modelo global para inferencia (online/offline).

Pipeline típico de ML centralizado



¿Cómo se "agrupan" los datos?

En la práctica, la centralización exige **unificar** datos heterogéneos:

- **Integración**: un mismo esquema lógico (tablas/feature store).
- **Normalización**: formatos, unidades, categorías, timestamps.
- **Identidades**: vincular eventos a usuario/dispositivo/cuenta (ID mapping).

Ventajas del enfoque centralizado

- Simplicidad operativa: un único lugar para entrenar, evaluar y versionar.
- Mejor aprovechamiento de hardware: GPUs compartidas, batch grande.
- Visibilidad global: métricas consistentes, depuración más directa.
- Rápida iteración: cambios en datos/feature store afectan a todo.

Pero... ¿por qué centralizar es un problema hoy?

- Datos cada vez más **sensibles** (salud, biometría, finanzas, localización).
- Regulación y gobernanza (p. ej. consentimiento, minimización, retención).
- Riesgo de **brecha** (single point of failure).
- Restricciones de **transferencia** entre entidades (inter-hospital, inter-empresa).

Amenazas y superficie de ataque en centralizado

- **Exfiltración** de datos: insiders, ataques al data lake, backups.
- **Reidentificación**: combinación de datasets aparentemente anónimos.
- **Acceso indebido**: permisos mal configurados, shadow copies.
- **Uso secundario**: entrenar para fines no consentidos.

Centralización: dilema de gobernanza

- ¿Quién es el **dueño** del dato?
- ¿Dónde se **almacena** (jurisdicción)?
- ¿Qué pasa si hay **múltiples organizaciones** y no confían entre sí?

Situación típica

Varios hospitales quieren un modelo común, pero no pueden mover historiales clínicos.

Resumen del bloque centralizado

- Centralizar simplifica el pipeline, pero concentra riesgo.
- Agrupar datos implica: integración, identidad, joins, feature store.
- Privacidad y cumplimiento se vuelven cuellos de botella.

Transición

Buscamos entrenar de forma colaborativa sin mover datos: **Federated Learning**.

Definición de Federated Learning (FL)

Definición informal

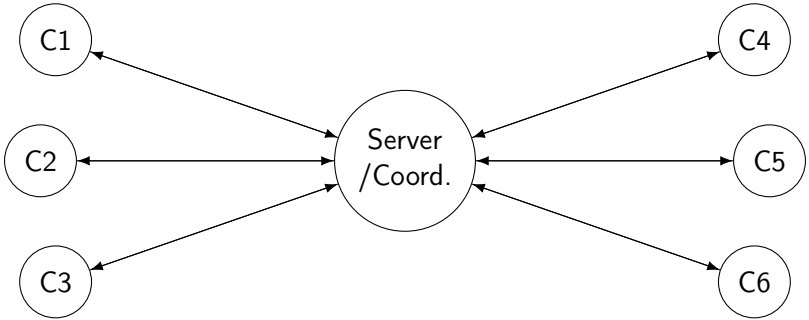
FL es un paradigma donde múltiples clientes entrenan localmente un modelo compartido y sólo envían **actualizaciones** (gradientes/pesos) a un coordinador que las **agrega**.

- Objetivo: un **modelo global** con datos distribuidos.
- Restricción: los datos crudos **no salen** del cliente.

¿Cómo se origina FL? (intuición histórica)

- Nace para entrenar modelos en **móviles** (teclado, recomendación, personalización) sin recopilar todos los datos en el servidor.
- Se populariza con el esquema **FedAvg**: promediar parámetros entrenados localmente.
- Después se expande a **salud**, **finanzas**, **industria**, **IoT**.

Esquema clásico: topología estrella



¿Qué se transmite en FL?

- **Pesos** del modelo (w) o **delta** de pesos (Δw).
- **Gradientes** (∇w) o **estadísticos** comprimidos.
- Metadatos: tamaño de dataset local n_k, métricas locales, estado del cliente.

Componentes de un sistema FL

- **Modelo**: arquitectura, inicialización, objetivos.
- **Protocolo de rondas**: cuántos clientes, cuántas épocas locales, etc.
- **Función de agregación** A: media ponderada, robusta, segura, etc.
- **Comunicación**: selección parcial, compresión, tolerancia a fallos.
- **Privacidad/Seguridad**: DP, secure aggregation, autenticación, etc.

Comparativa rápida: centralizado vs FL

	Centralizado	Federated Learning
Datos	Se mueven al servidor	Permanecen localmente
Entrenamiento	Un único sitio	Distribuido (local + agregación)
Riesgo de brecha	Alto (concentración)	Menor en datos crudos (no cero)
Heterogeneidad	Más controlable	Común: no-IID, dispositivos distintos
Comunicación	Baja (interna)	Crítica (red, compresión)

¿Qué problema "optimiza" FL?

Queremos minimizar una pérdida global sin acceso centralizado a datos:

$$\min_{w} \sum_{k=1}^{K} \frac{n_k}{N} \mathcal{L}_k(w) \qquad \text{donde} \qquad N = \sum_{k=1}^{K} n_k$$

- k identifica un cliente/silo; $\mathcal{L}_k$ es la pérdida en datos locales.
- Si los datos son no-IID, las $\mathcal{L}_k$ pueden ser muy diferentes.

Vista general de una ronda FL

1. El servidor tiene un modelo global w^t.
2. Selecciona clientes $\mathcal{S}_t$ y **envía** w^t.
3. Cada cliente entrena localmente y produce w_k^t (o Δw_k^t).
4. Los clientes **mandan** su update al servidor.
5. El agregador calcula $w^{t+1} = A(\{w_k^t\})$.
6. El servidor **manda** el nuevo modelo global w^{t+1}.

Paso 0: inicialización

- Elegir arquitectura y objetivo (clasificación, regresión, NLP, etc.).
- Inicializar w^0 (random o preentrenado).
- Definir hiperparámetros:
  - rondas T, fracción de clientes por ronda,
  - épocas locales E, batch size, learning rate,
  - compresión, clipping, etc.

Paso 1: selección de clientes

- Muestreo uniforme o ponderado (por disponibilidad, calidad, tamaño n_k).
- Restricciones: conectividad, carga, fairness, disponibilidad.

Nota

En cross-device es típico muestrear pocos clientes de muchos; en cross-silo, casi todos.

Paso 2: distribución del modelo global

- El servidor envía w^t (posiblemente comprimido o cuantizado).
- Se puede enviar también:
  - configuración de entrenamiento (lr, epochs),
  - semillas, máscaras, o capas congeladas,
  - reglas de validación local.

Paso 3: entrenamiento local (cliente)

Cada cliente k optimiza con sus datos locales:

$$w_k^t \leftarrow \text{LocalTrain}(w^t;\ D_k,\ E,\ \eta)$$

- Normalmente SGD/Adam con E épocas sobre D_k.
- El cliente puede devolver:
  - parámetros w_k^t, o
  - update $\Delta w_k^t = w_k^t - w^t$.

Paso 4: envío de actualizaciones

- Los clientes envían su update al servidor.
- Se puede añadir:
  - compresión (top-k, cuantización),
  - clipping,
  - mecanismos de privacidad (p. ej. ruido DP),
  - cifrado (secure aggregation).

¿Qué es un "update"?

Dos formas equivalentes en muchas implementaciones:

- Enviar w_k^t y agregar parámetros.
- Enviar Δw_k^t y aplicar:

$$w^{t+1} = w^t + \sum_k \alpha_k \Delta w_k^t$$

Paso 5: agregación en el servidor

- Entrada: $\{(w_k^t, n_k)\}_{k \in \mathcal{S}_t}$ o $\{(\Delta w_k^t, n_k)\}$.
- El agregador aplica $A(\cdot)$:
  - **Promedio** (FedAvg),
  - **Robusto** (mediana, trimmed mean, etc.),
  - **Seguro** (sobre valores cifrados/comprometidos),
  - **Mixto** (robust + verifiable, etc.).

La función de agregación $A(\cdot)$

Rol

Combinar updates locales en un update global **estable** y **eficiente**.

Ejemplo (media ponderada):

$$w^{t+1} = \sum_{k \in \mathcal{S}_t} \frac{n_k}{\sum_{j \in \mathcal{S}_t} n_j} \, w_k^t$$

- $\mathcal{S}_t$: subconjunto de clientes seleccionados en la ronda t.
- La elección de A condiciona: convergencia, robustez y privacidad.

Paso 6: broadcast del nuevo modelo

- Se genera w^{t+1} y se envía a los clientes de la siguiente ronda.
- Se repite hasta $t = T$ o convergencia.

Salida final

Modelo global w^T + (opcional) modelos personalizados por cliente.

Pseudocódigo: FedAvg (esqueleto)

```
Server:
  init w0
  for t = 0..T-1:
    St <- sample_clients()
    send w_t to clients in St
    receive {w_k, n_k} from k in St
    w_{t+1} <- sum_k (n_k / sum_j n_j) * w_k

Client k:
  receive w_t
  w <- w_t
  for e = 1..E:
    w <- w - eta * grad(L_k(w))
  send (w, n_k)
```

Taxonomía 1: Cross-device vs Cross-silo

- **Cross-device**: muchos clientes (móviles/IoT), pocos datos cada uno, alta volatilidad.
- **Cross-silo**: pocos clientes (organizaciones), muchos datos por cliente, infraestructura estable.

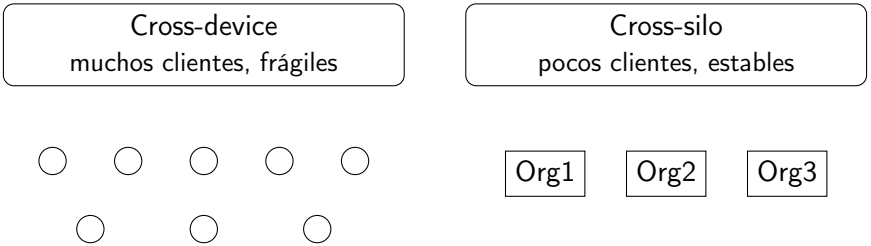
Cross-device FL: características

- K enorme (millones), muestreo pequeño por ronda.
- Clientes poco fiables: batería, red, desconexiones.
- Datos muy personales (usuario) y altamente no-IID.
- Comunicación/energía: restricciones severas $\Rightarrow$ compresión y participación parcial.

Cross-silo FL: características

- K pequeño (p. ej. 5–50 hospitales/empresas).
- Clientes con servidores dedicados, conectividad estable.
- Más factible usar criptografía pesada, auditoría y coordinación.
- Interesa mucho el **cumplimiento** y acuerdos entre entidades.

Ejemplo visual: cross-device vs cross-silo



Taxonomía 2: Centralized FL vs Distributed FL

- **Centralized FL**: un servidor coordina y agrega (topología estrella).
- **Distributed FL**: no hay único agregador; coordinación P2P o jerárquica.

Centralized FL (lo más común)

- Pro: simple, eficiente, fácil de desplegar.
- Contra: el servidor es punto central (confianza, fallo, objetivo de ataque).

Nota

La seguridad suele reforzarse con: secure aggregation, TEEs, auditoría, ZK proofs, etc.

Distributed FL: variantes

- **Peer-to-peer**: clientes intercambian y consensúan updates (gossip).
- **Jerárquico**: agregadores locales $\rightarrow$ agregador regional $\rightarrow$ global.
- **Multi-server**: varios coordinadores por zona/cluster.

Trade-off

Menos dependencia de un servidor, pero más complejidad y coordinación.

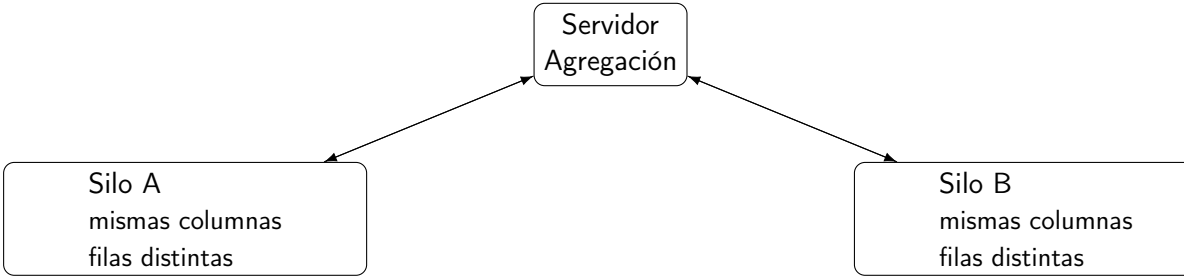
Taxonomía 3: Horizontal FL vs Vertical FL

- **Horizontal FL (HFL)**: mismos features, diferentes muestras.
- **Vertical FL (VFL)**: mismas muestras (o solapadas), diferentes features.

Horizontal FL (HFL)

- Caso típico: varios hospitales con la **misma estructura** de historia clínica.
- Cada silo tiene pacientes distintos $\Rightarrow$ se entrena un modelo común.
- Suele encajar bien con FedAvg y variantes.

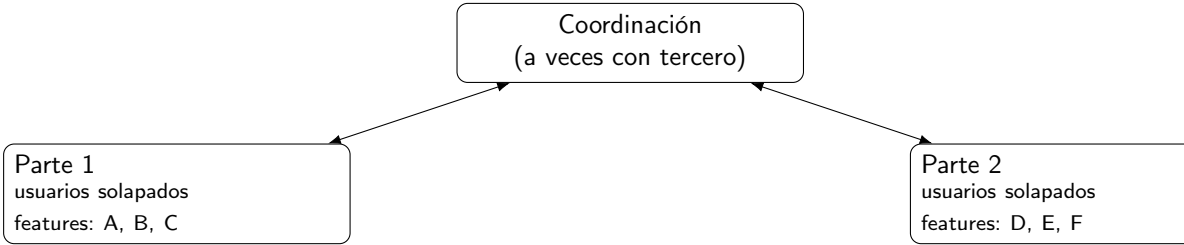
Diagrama HFL



Vertical FL (VFL)

- Diferentes entidades tienen **distintas columnas** para los mismos individuos.
- Ejemplo: banco (finanzas) + telco (movilidad) sobre usuarios solapados.
- Requiere emparejar identidades (entity resolution) y protocolos más complejos.

Diagrama VFL (conceptual)



- En VFL es común dividir el modelo (split learning / secure computation).

¿Y si no hay solapamiento perfecto?

- En VFL real suele haber solapamiento parcial de identidades.
- Aparecen técnicas de **federated transfer learning** (mención breve).

Resumen de tipologías (tabla)

Eje	Tipo	Rasgo típico
Participantes	Cross-device	muchos, frágiles, datos personales
	Cross-silo	pocos, estables, multi-organización
Arquitectura	Centralized FL	agregador único
	Distributed FL	P2P / jerárquico
Partición datos	Horizontal FL	mismas features, distintas muestras
	Vertical FL	mismas muestras, distintas features

Retos de FL (vista previa)

- **Estadística**: no-IID, desbalanceo, drift, personalización.
- **Sistemas**: comunicación cara, dispositivos heterogéneos, fallos.
- **Seguridad**: clientes maliciosos, poisoning, backdoors.
- **Privacidad**: leakage por updates, auditoría, cumplimiento.

¿Qué puede salir mal? (intuición)

- **no-IID**: cada cliente ve distribuciones diferentes.
- **stragglers**: clientes lentos retrasan rondas.
- **updates ruidosos**: pocos datos, mala calidad.
- **adversarios**: poisoning/backdoor.
- **leakage**: updates revelan información.

Desafíos principales en FL

- **Heterogeneidad de datos** (no-IID): distribuciones distintas por cliente.
- **Privacidad** (ataques de inferencia): lo que revelan los updates.
- **Seguridad** (ataques de envenenamiento): updates manipulados para degradar o insertar backdoors.

Idea guía

Tras cada problema: **mitigaciones típicas** (agregación / rebalanceo / DP / robust aggregation).

Problema 1: heterogeneidad de datos (no-IID)

En FL, cada cliente k tiene su propia distribución $P_k(x, y)$.

- **Label skew**: proporciones de clases distintas.
- **Feature skew**: sensores/mediciones diferentes; distinto ruido/calibración.
- **Quantity skew**: tamaños n_k muy desiguales.
- **Concept drift**: la distribución cambia con el tiempo.

¿Qué efecto tiene el no-IID?

- **Client drift**: cada cliente optimiza hacia un mínimo local distinto.
- Convergencia más lenta / inestable; oscilaciones entre rondas.
- Peor generalización para subpoblaciones raras.

Intuición

FedAvg funciona muy bien si los P_k son parecidos; cuando difieren mucho, hay conflicto entre gradientes.

Soluciones al no-IID (I): agregación y variantes

Mitigaciones basadas en **cómo agregamos**:

- **Ponderaciones adaptativas**: no sólo por n_k (calidad, estabilidad, similitud).
- **Agregación por clusters**: agrupar clientes similares y promediar dentro de cluster.
- **Regularización / corrección**: reducir drift con términos de proximidad al global (idea tipo "proximal").
- **Momentum en servidor**: suaviza oscilaciones entre rondas.

Soluciones al no-IID (II): rebalanceo y muestreo de clientes

Mitigaciones basadas en **quién participa** y con qué peso:

- **Muestreo estratificado**: asegurar presencia de subpoblaciones/clases.
- **Rebalanceo**: limitar el dominio de clientes gigantes (cap de peso / cap de muestras).
- **Reweighting**: ajustar α_k para evitar que unos pocos dominen.
- **Curriculum**: primero clientes más estables, luego más heterogéneos.

Resultado esperado

Mejor estabilidad y fairness entre clientes/subpoblaciones.

Problema 2: ataques de inferencia (privacidad)

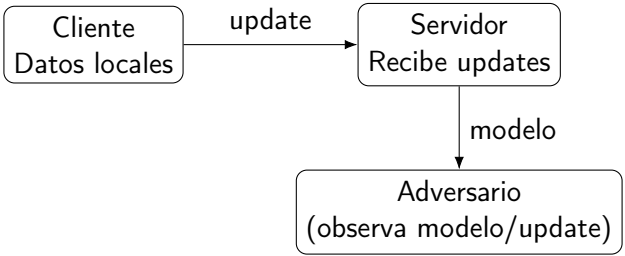
Aunque los datos no se suben, los **updates** pueden filtrar información:

- **Membership inference**: inferir si un ejemplo estuvo en entrenamiento.
- **Property inference**: inferir una propiedad del dataset local (p. ej. atributo sensible).
- **Reconstruction / gradient leakage**: reconstruir ejemplos a partir de gradientes.

Clave

Privacidad en FL no es automática: depende de qué se comparte y cómo.

Ejemplo conceptual: membership inference



- El adversario prueba un ejemplo z y estima si el modelo "lo recuerda".
- Mayor riesgo con modelos muy expresivos y datasets pequeños.

Solución a privacidad: Differential Privacy (DP)

Idea

Asegurar que la contribución de un individuo cambia poco el resultado, añadiendo **ruido calibrado**.

Patrón típico (DP-SGD):

$$\text{clip}(\nabla w) \quad \rightarrow \quad \text{sumar ruido} \quad \rightarrow \quad \text{actualizar}$$

- **Clipping**: acota sensibilidad de gradientes/updates.
- **Ruido**: (p. ej. Gaussiano) para obtener garantías (ε, δ).

DP en FL: dónde se aplica

- **Local DP**: el cliente añade ruido antes de enviar (más fuerte, pero más pérdida de utilidad).
- **Central DP**: el servidor añade ruido tras agregar (mejor utilidad, requiere confianza/secure agg).

Trade-off

Más privacidad $\Rightarrow$ más ruido $\Rightarrow$ posible caída de accuracy.

Problema 3: ataques de envenenamiento (poisoning)

Ataques para degradar el modelo o insertar comportamiento malicioso:

- **Data poisoning**: contaminar datos locales (labels, ejemplos).
- **Model/update poisoning**: enviar updates manipulados (dirección o magnitud).
- **Backdoor**: mantener accuracy normal pero fallar en un trigger.
- **Sybil**: un atacante simula muchos clientes para ganar peso.

¿Por qué FL es vulnerable a poisoning?

- El servidor no ve los datos: difícil auditar si un update es "legítimo".
- Participación parcial: el atacante puede aparecer de forma intermitente.
- Agregación lineal: una pequeña fracción puede sesgar si no hay defensas.

Consecuencia

Se necesita robustez en la **función de agregación** y/o filtrado.

Solución a seguridad: agregación robusta (idea)

En vez de promediar sin más, usar $A(\cdot)$ resistente a outliers/maliciosos:

- **Mediana / trimmed mean** (coordenada a coordenada): reduce influencia de extremos.
- **Krum / Multi-Krum**: elige updates cercanos a la mayoría.
- **Norm bounding / clipping**: limita magnitud del update.
- **Filtrado por similitud**: detectar y descartar updates muy desviados.

Agregación robusta: mini-comparativa

Familia	Intuición	Riesgo / coste
Mediana/Trimmed	descarta extremos	puede sesgar si hay no-IID fuerte
Krum	escoge "centro"	coste computacional (distancias)
Clipping	limita impacto	no detecta direcciones coordinadas
Filtrado	detecta anomalías	umbrales/diseño delicado

Resumen: problema → mitigaciones

Problema	Soluciones típicas
Heterogeneidad (no-IID)	Agregación adaptativa/cluster + rebalanceo/muestreo de clientes
Privacidad (inferencia)	DP (clipping + ruido); (opcional) secure aggregation como complemen
Seguridad (poisoning)	Funciones de agregación robustas + filtrado/detección de outliers

Idea final

En FL siempre hay **trade-offs**: utilidad vs privacidad vs robustez vs coste.

¿Cuándo tiene sentido usar FL?

- No puedes centralizar por privacidad, regulación o competencia.
- Quieres aprendizaje continuo en edge con datos locales.
- Te interesa colaboración multi-entidad sin compartir datos crudos.

Checklist rápido

Datos distribuidos + valor de un modelo común + restricciones de centralización $\Rightarrow$ FL candidato.

Limitaciones (honestas) de la “intro”

- FL no garantiza privacidad por sí solo (DP/secure agg suelen ser necesarios).
- Rendimiento puede bajar bajo no-IID (mitigar con agregación/selección/personalización).
- Seguridad no es gratuita: poisoning requiere defensas (robust aggregation, filtrado, etc.).

Mini-resumen final

- Centralizado: datos se agrupan en un repositorio, con ventajas pero alto riesgo de privacidad.
- FL: el modelo se entrena **donde están los datos** y se agrega en rondas.
- El corazón del sistema es el ciclo: **entrenar–mandar–agregar–mandar**.
- Tipologías: cross-device/cross-silo, centralized/distributed, horizontal/vertical.
- Desafíos: no-IID, inferencia, poisoning $\rightarrow$ mitigaciones (agregación/DP/robust agg).

Preguntas para discusión

- ¿Qué es más crítico en tu dominio: privacidad, robustez o eficiencia?
- ¿Tus datos están más cerca de cross-device o cross-silo?
- ¿Tu problema es HFL o VFL? ¿Cómo afecta al diseño del protocolo?
- ¿Qué mitigación priorizarías: rebalanceo, DP o robust aggregation?

Fin